

# Réseaux de neurones profonds

## Réseaux Convolutionels

## Résumé

Ce document constitue une introduction aux réseaux de neurones convolutionnels. Il contient les éléments nécessaires à la compréhension des convolutions et à l'utilisation des réseaux convolutionnels en pratique.

## Table des matières

|          |                                                                                    |           |
|----------|------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                | <b>4</b>  |
| 1.1      | Notion de convolution . . . . .                                                    | 4         |
| 1.1.1    | Convolution 1D, formule et illustration . . . . .                                  | 4         |
| 1.1.2    | Convolution 2D : formule et illustration . . . . .                                 | 6         |
| 1.1.3    | Exemple d'application . . . . .                                                    | 6         |
| 1.2      | Motivation des réseaux de neurones convolutionnels . . . . .                       | 9         |
| 1.2.1    | Utilisation de connexions "creuses" : réduire les ressources nécessaires . . . . . | 9         |
| 1.2.2    | Partage de paramètres et invariance . . . . .                                      | 10        |
| <b>2</b> | <b>Implémentation dans un réseau de neurones</b>                                   | <b>11</b> |
| 2.1      | Implémentation en PyTorch . . . . .                                                | 11        |
| 2.1.1    | in channels . . . . .                                                              | 11        |
| 2.1.2    | out channels . . . . .                                                             | 11        |
| 2.1.3    | Kernel Size . . . . .                                                              | 11        |
| 2.1.4    | Stride . . . . .                                                                   | 12        |
| 2.1.5    | Padding . . . . .                                                                  | 12        |
| 2.1.6    | Dilatation . . . . .                                                               | 13        |
| 2.1.7    | Groups . . . . .                                                                   | 13        |
| 2.2      | Calculer les dimensions de sortie . . . . .                                        | 14        |
| 2.2.1    | Exemple 1 . . . . .                                                                | 14        |
| 2.2.2    | Exemple 2 . . . . .                                                                | 16        |
| 2.3      | Nombre de paramètres . . . . .                                                     | 17        |
| 2.4      | De l'usage des biais . . . . .                                                     | 18        |
| <b>3</b> | <b>Pooling</b>                                                                     | <b>18</b> |
| 3.1      | Introduction et motivations . . . . .                                              | 18        |
| 3.2      | Implémentation et taille de données . . . . .                                      | 19        |
| 3.3      | Différents types de pooling . . . . .                                              | 19        |
| 3.3.1    | Max pooling . . . . .                                                              | 20        |
| 3.3.2    | Average pooling . . . . .                                                          | 20        |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 3.3.3    | Lp pooling . . . . .                                             | 21        |
| 3.3.4    | Pooling par réseau de neurones . . . . .                         | 21        |
| 3.4      | Pooling, une opération multi-dimensionnelle . . . . .            | 21        |
| <b>4</b> | <b>Exemple d'application</b>                                     | <b>22</b> |
| 4.1      | Classification d'images MNIST . . . . .                          | 22        |
| 4.2      | Architectures classiques . . . . .                               | 25        |
| 4.2.1    | Temporal- Delay Neural Network (TDNN) . . . . .                  | 25        |
| 4.2.2    | Temporal Convolutional Network (TCN) . . . . .                   | 26        |
| 4.2.3    | AlexNet . . . . .                                                | 26        |
| 4.2.4    | ResNet . . . . .                                                 | 27        |
| 4.2.5    | MobileNet . . . . .                                              | 29        |
| 4.2.6    | U-Net . . . . .                                                  | 36        |
| 4.3      | Convolution par groupes . . . . .                                | 41        |
| 4.4      | Implémenter une convolution <i>Depthwise</i> . . . . .           | 43        |
| 4.5      | Implémenter une convolution <i>Depthwise Separable</i> . . . . . | 44        |
|          | <b>Bibliography</b>                                              | <b>45</b> |

## 1 Introduction

### 1.1 Notion de convolution

#### 1.1.1 Convolution 1D, formule et illustration

Soient deux fonctions mathématiques discrètes  $f$  et  $g$ . La convolution de  $f$  et  $g$ , notée  $(f * g)$ , est donnée par l'équation :

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[k]g[n - k] \quad (1)$$

Qui est également équivalent à :

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[n - k]g[k] \quad (2)$$

Dans le cas où la fonction  $g$  a un ensemble de définition borné sur  $[-M; M]$ , c'est-à-dire qu'il s'agit d'une séquence de longueur finie, alors :

$$(f * g)[n] = \sum_{k=-M}^M f[n - k]g[k] \quad (3)$$

La figure 1 illustre le calcul de la convolution discrète à 1 dimension.

#### Note

L'ensemble de définition de la fonction  $f * g$  est l'ensemble de définition de la fonction  $f$  réduit aux deux extrémités par la moitié de l'ensemble de définition de  $g$ .

Remarquez que le maximum de la fonction de convolution est obtenu lorsque le filtre est parfaitement aligné avec la *porte* de la fonction  $f$ . L'utilisation de convolution avec un filtre déterminé permet de détecter des motifs dans un signal ( *patterns* ).

#### Définitions

Dans l'exemple ci-dessus, la fonction  $g$  est appelée filtre ou **kernel** en anglais.



FIGURE 1 – Illustration d’une convolution discrète à 1-dimension convolution Notez que la convolution n’est pas définie sur le même interval que la fonction  $f$ .

### 1.1.2 Convolution 2D : formule et illustration

Le concept de convolution peut être appliqué à des signaux à plusieurs dimensions. La convolution en 2 dimension est une opération bien connue en traitement des images qui est utilisée pour de nombreuses applications telles que la détection de contours.

Soit deux signaux discrets,  $f$  et  $g$ , à deux dimensions et définis sur  $\mathbb{Z} \times \mathbb{Z}$ , alors la convolution  $f * g$  est donnée par :

$$h(x, y) = (g * f)(x, y) = \sum_m \sum_n g(x - m, y - n) f(m, n) \quad (4)$$

La convolution 2D peut être décomposée en 2 convolutions 1D mais ce point n'est pas détaillé dans ce cours. La figure 2 illustre la convolution d'une image (signal discret à 2 dimensions) avec un filtre de dimension  $3 \times 3$

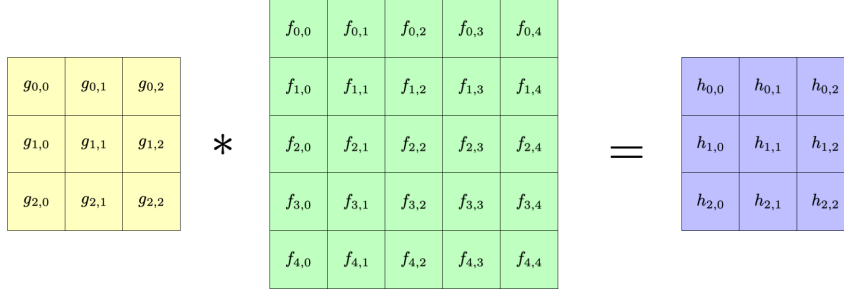


FIGURE 2 – Convolution 2D entre une image de taille  $5 \times 5$  et un filtre de taille  $3 \times 3$ .

avec, dans cet exemple :

$$h_{i,j} = \sum_{n=0}^2 \sum_{m=0}^2 g_{n,m} \times f_{i+n,j+m} \quad (5)$$

### 1.1.3 Exemple d'application

En pratique le traitement est appliqué sur l'ensemble des données d'entrée en se déplaçant d'un pas *stride* fixe. Si les dimensions des données d'entrée ne sont pas des multiples des dimensions du filtre, il est possible que toutes les données d'entrée ne soient pas utilisées pour produire la sortie.

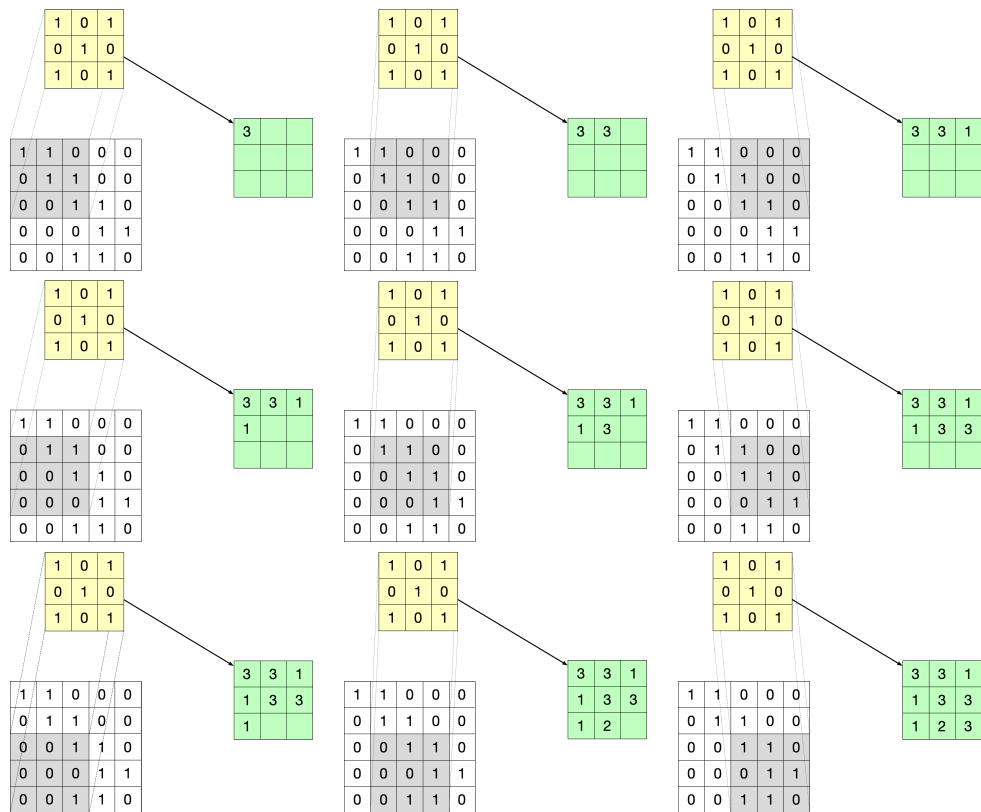
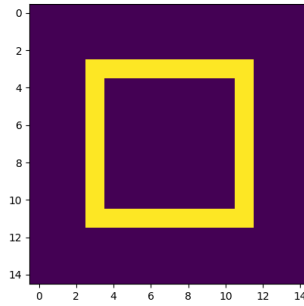
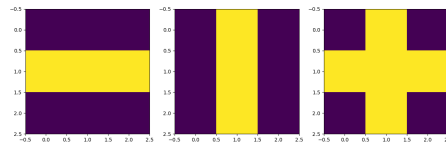


FIGURE 3 – Illustration d'une convolution discrète à 2-dimension convolution. Notez que l'image résultant de la convolution n'a pas la même dimension que l'originale.

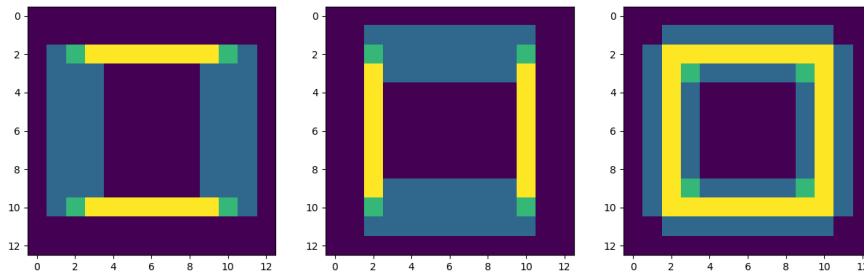
En traitement du signal digital et traitement des images, les convolutions (qui peuvent être implémentées comme des corrélations) sont depuis longtemps utilisées pour diverses applications comme la détection de contour (<https://kobiso.github.io/research/research-cnn/>). Un exemple est donné par la figure 4.



(a) Image à traiter



(b) Filtres convolutionnels



(c) Sortie des convolutions

FIGURE 4 – L’application de convolutions appliquées à l’image du dessus en utilisant les 3 filtres produit les 3 sorties de la ligne du bas.



## 1.2 Motivation des réseaux de neurones convolutionnels

L'utilisation des réseaux convolutionnels est motivée par plusieurs facteurs :

- l'utilisation de connections "creuses" entre les couches
- le partage de paramètres
- l'invariance

### 1.2.1 Utilisation de connections "creuses" : réduire les ressources nécessaires

Les réseaux de neurones "classiques" dans lesquels tous les neurones d'une couche sont reliés à tous les neurones de la couche suivante ont un très grand nombre de paramètres qui est dépendant de la dimension des données d'entrée de chaque couche. Un très grand nombre de paramètres implique une utilisation importante de mémoire vive et un temps de calcul élevé.

Exemple, pour traiter une image en noir et blanc de dimension  $28 \times 28$ , une couche de neurones à 1000 neurones contient  $28 \times 28 \times 1000 + 1000 = 785000$  paramètres. Traiter une image couleurs de dimensions réelles : par exemple à 24 millions de pixels nécessite  $72 \cdot 10^9$  paramètres. Même sur les machines actuelles, ce serait techniquement difficile.

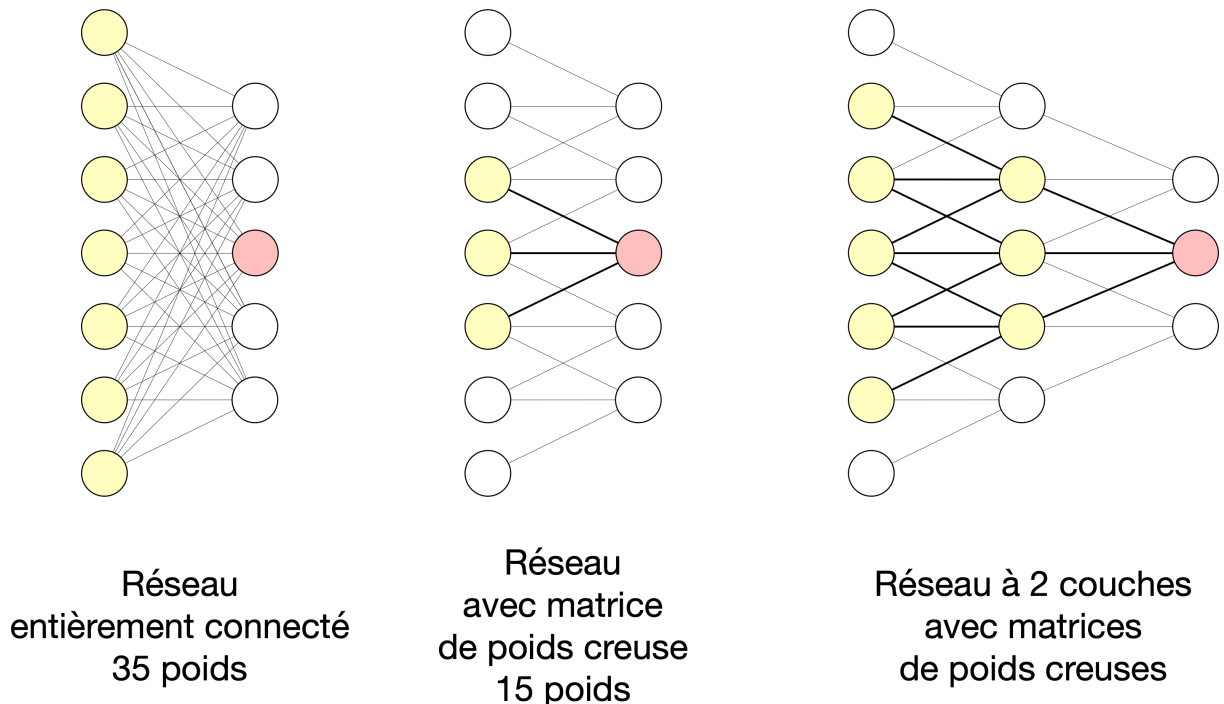


FIGURE 5 – Les réseaux convolutionnels ne nécessitent pas que chaque neurone traite toutes les entrées. Chaque "filtre" ne traite qu'une information locale.

Dans l'exemple de la Figure 5, la figure de gauche représente un réseau dont les couches sont entièrement connectées, nécessitant 35 paramètres de poids. La sortie du neurone rouge prend

en compte l'intégralité de la couche d'entrée. La figure du milieu représente un réseau dont la matrice de poids est creuse. Le neurone rouge ne reçoit en entrée qu'une partie locale du vecteurs d'entrée correspondant aux entrées colorées en jaune. La sous-partie de l'entrée, prise en compte par le neurone rouge est appelée *receptive field* de ce neurone. La figure de droite illustre l'effet de la profondeur du réseau sur le *receptive field*, plus le réseau est profond et plus les couches intègrent toute l'information de l'entrée. Les réseaux convolutionnels permettent ainsi de traiter des entrées de très grande dimension en réduisant le nombre de paramètres et en exploitant une information locale.

Inspiré des méthodes de traitement du signal (filtrage) et de traitement d'images, les réseaux convolutionnels permettent de traiter des données d'entrée de grande dimension en réduisant le nombre de paramètres.

### 1.2.2 Partage de paramètres et invariance

Dans un réseau de neurones traditionnel, un paramètre est utilisé exactement une fois lors de la phase *forward*. Le poids est multiplié à la variable d'entrée et n'est jamais réutilisé ensuite. Dans le cas de réseaux convolutionnels, le poids d'un filtre est utilisé pour traiter toutes les zones "locales" de l'entrée (sauf peut être les zones de "bordure"). Ainsi, au lieu d'apprendre un jeu de paramètre pour chaque partie du signal d'entrée, le réseau apprend un jeu de paramètre partagé sur l'ensemble des zones de cette entrée. Le temps de calcul n'en n'est pas augmenté mais le nombre de paramètres est grandement réduit.

Les données à traiter par des réseaux de neurones sont structurées. La plupart du temps, les données proches (spatialement ou temporellement) sont corrélées. De plus, dans dans nombreux cas, l'information présente dans le signal peut se trouver à différents endroits, différents moments. Il existe une invariance spatial et temporel de l'information. Une analyse locale des données est alors intéressante.

Très souvent, les phénomènes complexes, dont la modélisation nécessite des réseaux de neurones profonds, sont composés de phénomènes plus petits et plus simples eux mêmes décomposables de la même manière. Les réseaux convolutionnels exploitent la structure des données

Dans le cas des réseaux de neurones convolutionnels, on parle d'invariance spatiale car le même traitement est appliqué à toutes les sous-parties de l'entrée. Ainsi, une même convolution appliquée localement sur toute une image produira la même sortie à chaque fois qu'un détail similaire apparaît sur l'image.

Dans le cas du traitement d'une série temporelle, le réseau produira la même sortie à chaque fois qu'un événement se répète dans le temps, faisant ainsi apparaître la structure du signal d'entrée. Les réseaux convolutionnels sont inspirés du cortex visuel et permettent une analyse structurée du signal.

## 2 Implémentation dans un réseau de neurones

Les réseaux convolutionnels sont implémentés dans toutes les bibliothèques d'apprentissage profond et font partie intégrantes des architectures pour de très nombreuses applications. Cette section présente l'implémentation disponible dans la bibliothèque PyTorch 2.4 à titre d'exemple.

### 2.1 Implémentation en PyTorch

Comme toutes les bibliothèques modernes pour l'apprentissage profond, Pytorch implémente des couches convolutionnelles 1D, 2D et 3D.

La documentation en ligne présente la formule de la convolution 1D comme suit :

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \star input(N_i, k) \quad (6)$$

où l'entrée a pour dimensions :  $(N, C_{in}, L)$  avec  $N$  la taille de batch,  $C_{in}$  le nombre de canaux d'entrée et  $L$  la longueur du signal et la sortie a pour dimensions  $(N, C_{out}, L_{out})$  avec  $C_{out}$  et  $L_{out}$  respectivement nombre de canaux de sortie et longueur de la séquence de sortie. L'opérateur  $\star$  est l'inter-corrélation.

Cette couche de neurones est définie par différents paramètres que sont : `in_channels`, `out_channels`, `kernel_size`, `stride`, `padding`, `dilation` et `groups`.

#### 2.1.1 in channels

Nombre de canaux d'entrée de la couche convolutionnelle.

#### 2.1.2 out channels

Nombre de canaux de sortie produits par la couche convolutionnelle, c'est le nombre de filtre qui seront appliqués à chaque canal de l'entrée.

#### 2.1.3 Kernel Size

La taille du filtre est un paramètre essentiel des couches convolutionnelles. Elle détermine la taille du *receptive field*, le contexte qui est vu par le réseau, ou en d'autres termes la taille de la fenêtre d'analyse, ou encore la taille du motif analysé. La taille du filtre influe sur la quantité de calcul à effectuer et sur la dimension de la sortie.

### 2.1.4 Stride

Le paramètre **stride** permet de définir le "pas" de déplacement du filtre sur la signal lors de la convolution. Par défaut,  $stride = 1$  comme illustrée sur la première ligne de la figure 6, mais il est possible, si le signal est redondant ou pour accélérer le traitement en réduisant la résolution de la sortie, de fixer un pas plus grand, illustré par la ligne inférieure de la figure 6 où le pas est fixée à 2. Notez que le pas de  $stride$  peut prendre des valeurs différentes selon la dimension de déplacement et que dans **Pytorch** le paramètre **stride** est un n-uplet (couple pour une convolution 2D) mais qu'un scalaire,  $k$ , peut être donné en paramètre. Dans ce cas, le paramètre est fixé implicitement comme un n-uplet de  $k$ . Par exemple en 2D, fixer le paramètre à  $k$  est identique à  $(k, k)$ .

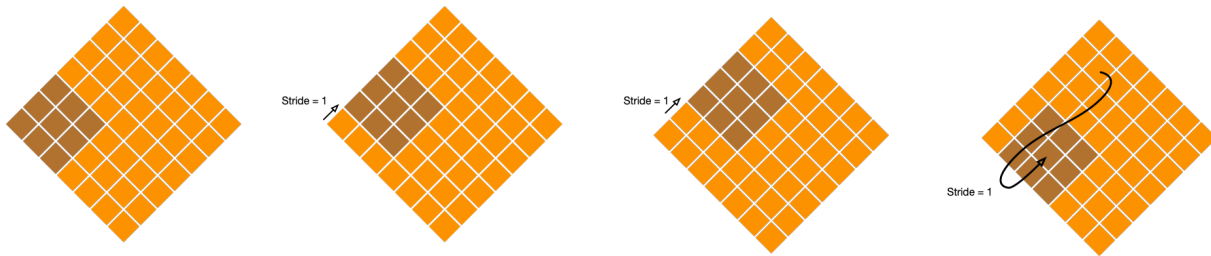


FIGURE 6 – Illustration de l'effet du paramètre  $stride$  sur une convolution 2D. Sur la première ligne, le pas est de 1, ou plutôt  $(1, 1)$  alors que sur la deuxième ligne il est de 2 (ou  $(2, 2)$ ).

### 2.1.5 Padding

Nous avons observé dans les exemples précédents que la convolution produit des sorties de dimensions plus petites que les entrées. Afin de préserver la dimension d'entrée, il est possible, en utilisant le paramètre de *padding* d'ajouter des valeurs au *bord* du signal d'entrée, afin d'augmenter artificiellement sa dimension et de produire une sortie de même taille que l'entrée réelle. L'utilisation de *padding* est illustrée par les figure 7 où l'on visualise l'augmentation de taille du signal d'entrée (dans ce cas un signal à 2 dimensions) et par la figure 8 où on visualise l'effet du *padding* sur la taille de la sortie : le carré bleu (sortie) à la même taille que le carré orange (entrée avant padding).

Les nouvelles variables d'entrée créées avec le padding peuvent prendre plusieurs valeurs :

'zeros' par défaut, les valeurs ajoutées sont des zéros.

'reflect' ajoute les valeurs d'entrée en miroir

'replicate' réplique la valeur du bord du signal dans cette direction

'circular' les valeurs sont répétées sur chaque dimension (les valeurs du début sont répétées à la fin et celle de la fin au début)

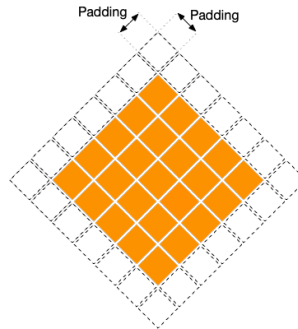


FIGURE 7 – Pour conserver la dimension de l'entrée, la dimension de celle-ci est artificiellement augmentée avec des valeurs prédéfinies. Avec un paramètre `padding = 1`, la taille dans chaque dimension est augmentée de 1.

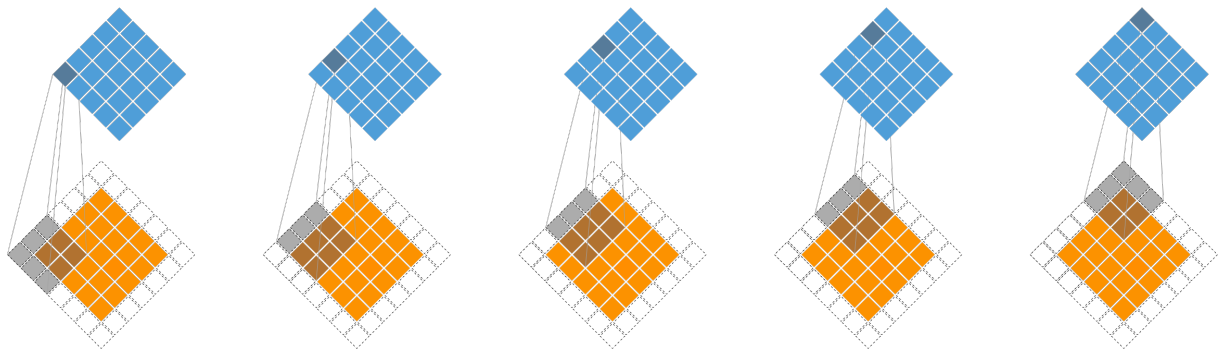


FIGURE 8 – L'utilisation d'une convolution 2D avec un filtre de dimension  $(3, 3)$  et un `padding` de 1 permet de produire une sortie de la même dimension que l'entrée.

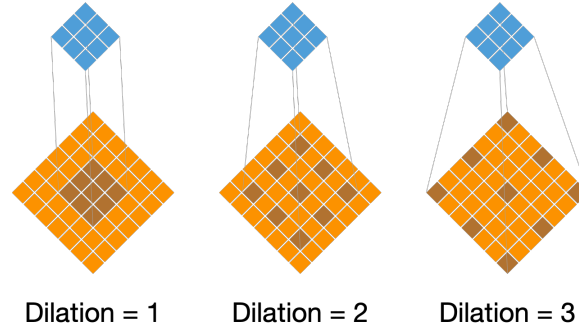
### 2.1.6 Dilatation

En pratique les données d'entrée sont souvent redondantes et les *pixels* de l'entrée, ou, plus généralement, les valeurs contiguës du signal d'entrée sont corrélées. On peut réduire la quantité de calcul nécessaire tout en tirant partie de cette corrélation en utilisant le paramètre de dilatation, ou *dilation* en anglais, qui permet de n'utiliser qu'une valeur sur  $N$  du signal d'entrée. Cet effet est illustré par la figure 9 avec une valeur de 2. L'utilisation de la dilatation permet d'augmenter la "surface" du *receptive field* sans augmenter la quantité de calcul à effectuer.

### 2.1.7 Groups

Le paramètre `groups` permet de différencier les convolutions appliquées sur les canaux d'entrée. Par exemple, pour une entrée à 100 canaux, l'utilisation de 5 groupes va diviser cette entrée en 5 groupes de 20 canaux auxquels seront appliqués des convolutions différentes.

Le nombre de canaux doit être un diviseur du nombre de canaux d'entrée et de canaux de sortie.

FIGURE 9 – Effet du paramètre de *dilation* sur une convolution 2D.

## 2.2 Calculer les dimensions de sortie

L'utilisation des couches convolutionnelles nécessite de bien maîtriser la dimension des données qui transitent entre les couches. La dimension du signal de sortie d'une couche convolutionnelle dépend des méta-paramètres de la couche : le **stride**, le **padding**, la **dilation** et la taille du filtre (**kernel\_size**).

Pour une convolution 2D, si les données d'entrée sont de dimension  $N, C_{in}, H, W$  avec  $N$  la taille du batch,  $C_{in}$  le nombre de canaux d'entrée, et  $H$  et  $W$  respectivement la hauteur et la largeur des échantillons, alors la dimension des données de sortie de cette couche est  $(N, C_{out}, H_{out}, W_{out})$  et les valeurs des nouvelles hauteurs et largeurs de ces sorties peuvent être calculée de la façon suivante :

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - dilation[0] \times (kernel\_size[0] - 1) - 1}{stride[0]} + 1 \right\rfloor \quad (7)$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times padding[1] - dilation[1] \times (kernel\_size[1] - 1) - 1}{stride[1]} + 1 \right\rfloor \quad (8)$$

### 2.2.1 Exemple 1

Calculer la taille des données de sorties pour :

- taille de l'entrée :  $(4, 4)$
- taille du filtre :  $(2, 3)$
- padding :  $(1, 1)$
- stride :  $(2, 2)$
- dilation :  $(1, 1)$

La dimension de la sortie est  $(H_{out}, W_{out}) = (3, 2)$

$$H_{out} = \frac{4 + 2 \times 1 - 1 \times (2 - 1) - 1}{2} + 1 \quad (9)$$

$$W_{out} = \frac{4 + 2 \times 1 - 1 \times (3 - 1) - 1}{2} + 1 \quad (10)$$

$$(11)$$

Le résultat est illustré par la figure 10.

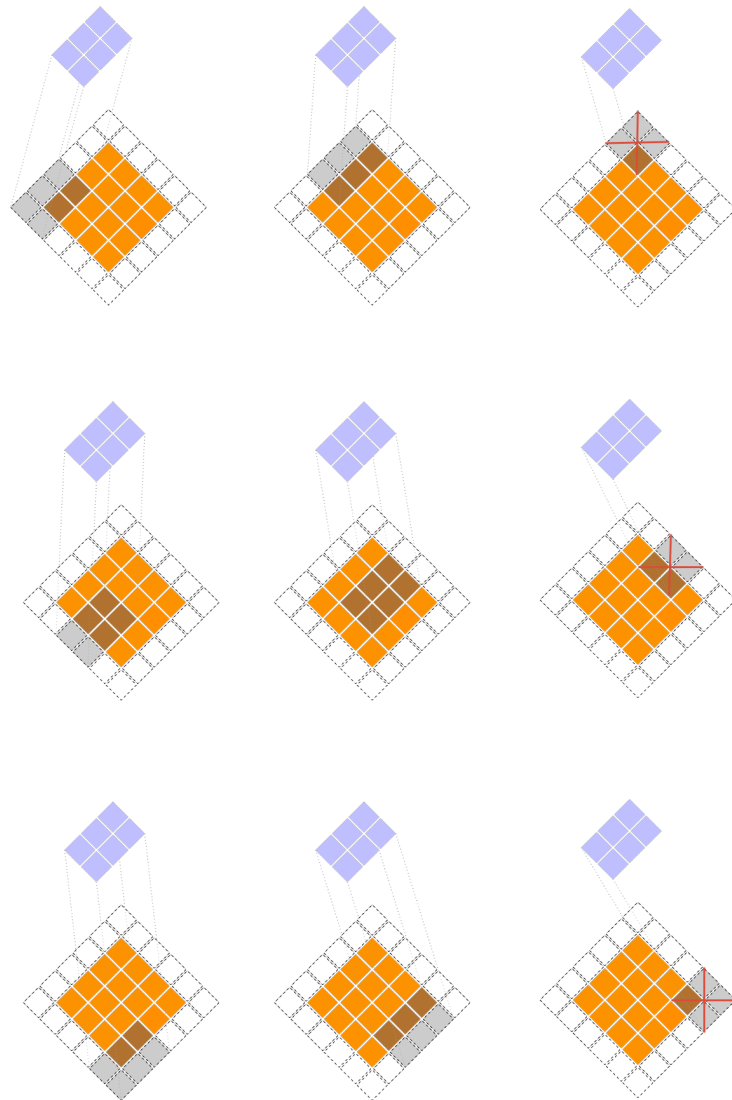


FIGURE 10 – Convolution d’une image de dimension (4, 4) par un filtre de dimension (2, 3) avec un padding de 1, une dilatation de 1 et un pas de 2. Dans la colonne de droite, le filtre ne peut pas être appliqué car il déborde de l’image. le résultat est donc de dimension (3, 2)

### 2.2.2 Exemple 2

Calculer la taille des données de sorties pour :

- taille de l'entrée :  $(7, 7)$
- taille du filtre :  $(2, 3)$
- padding :  $(1, 2)$
- stride :  $(1, 2)$
- dilation :  $(2, 2)$

La dimension de la sortie est  $(H_{out}, W_{out}) = (7, 4)$

$$H_{out} = \frac{7 + 2 \times 1 - 2 \times (2 - 1) - 1}{1} + 1 \quad (12)$$

$$W_{out} = \frac{7 + 2 \times 2 - 2 \times (3 - 1) - 1}{2} + 1 \quad (13)$$

$$(14)$$

Le résultat est illustré par la figure 11.

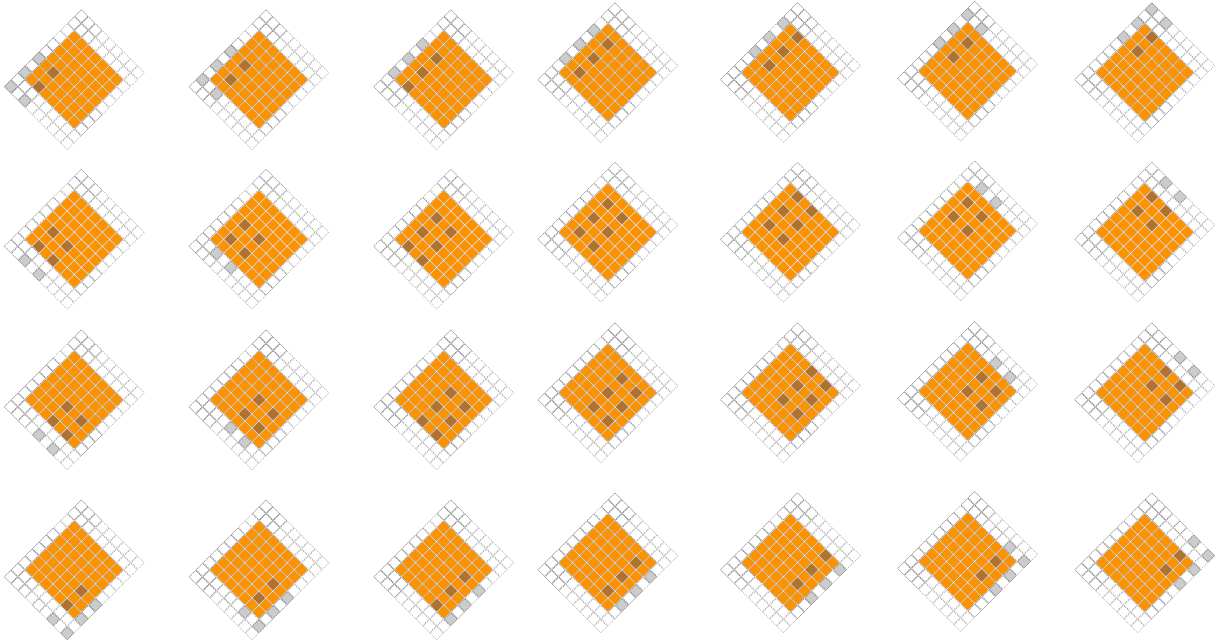


FIGURE 11 – Convolution d'une image de dimension  $(7, 7)$  par un filtre de dimension  $(2, 3)$  avec un padding de  $(1, 2)$ , une dilatation de  $(2, 2)$  et un pas de  $(1, 2)$ . Le résultat est donc de dimension  $(7, 4)$



### 2.3 Nombre de paramètres

Le tenseur de poids du module est de taille  $(out\_channels, \frac{in\_channels}{groups}, kernel\_size[0], kernel\_size[1])$ . Et les valeurs sont initialisées aléatoirement en suivant une distribution uniforme telle que  $\mathcal{U}(-\sqrt{k}, \sqrt{k})$  avec

$$k = \frac{groups}{C_{in} \times \prod_{i=0}^1 kernel\_size[i]} \quad (15)$$

Le tenseur de biais du module est de taille  $(out\_channels, \frac{in\_channels}{groups}, kernel\_size[0], kernel\_size[1])$  où les paramètres de biais sont initialisés aléatoirement selon une distribution  $\mathcal{U}(-\sqrt{k}, \sqrt{k})$  avec :

$$k = \frac{groups}{C_{in} \times \prod_{i=0}^1 kernel\_size[i]} \quad (16)$$

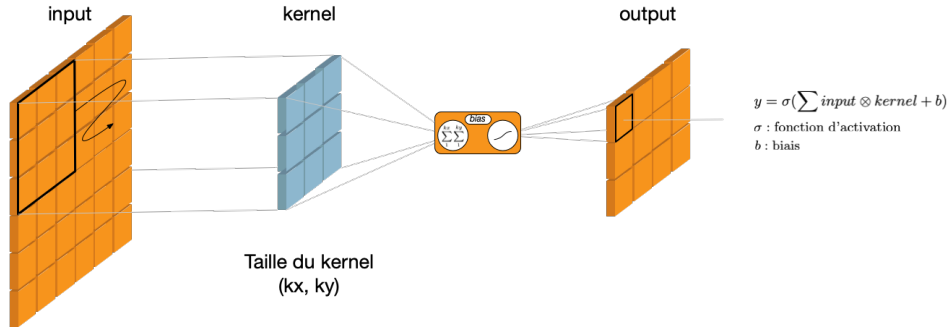


FIGURE 12 – Architecture d’une couche convolutionnelle 2D à 1 unique filtre de dimension  $(kx, ky)$ .

La couche convolutionnelle telle que représentée par la figure 12 contient  $kx \times ky + 1$  paramètres.

En ajoutant des filtres on obtient l’architecture suivante : La couche convolutionnelle telle

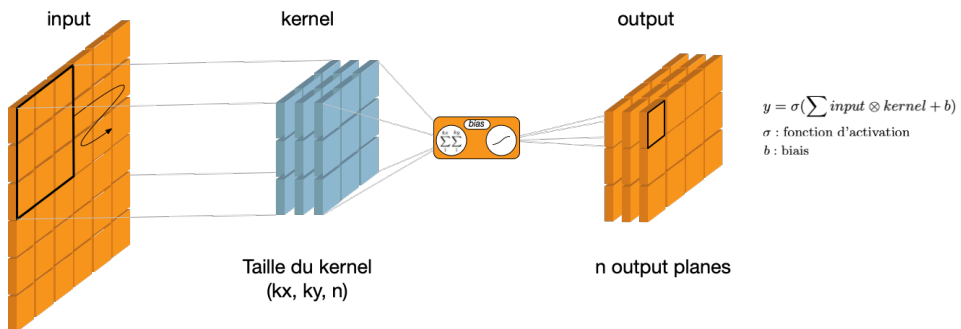


FIGURE 13 – Architecture d’une couche convolutionnelle 2D à 3 filtres de dimension  $(kx, ky)$ .

que représentée par la figure 12 contient  $n \times kx \times ky + n$  paramètres.

### 2.4 De l'usage des biais

Dans les réseaux récents, la plupart des couches neuronales sont suivies d'une forme de normalisation des données de sortie. La plus courante est certainement la "BatchNorm" ( `torch.nn.BatchNorm2d`) qui inclue un biais. Ainsi le biais de cette couche de normalisation s'ajoute-t'il à celui de la couche neuronale dans le cas où les deux biais précèdent la fonction d'activation. il est donc plus économique de supprimer le biais de la couche neuronale `bias=False` lors de l'initialisation de la couche.

C'est souvent le cas pour les réseaux convolutionnels pour lesquels l'enchaînement des couches est souvent : `Conv -> BatchNorm -> ReLU`

Exemple d'une architecture où le biais peut être supprimé :

```
nn.Conv2d(h_dim, h_dim, 3, stride, 1, groups=h_dim, bias=False),
nn.BatchNorm2d(h_dim),
nn.ReLU6(inplace=True),
```

Exemple d'un bloc où le biais ne peut pas être supprimé :

```
nn.Linear(64, 32),
nn.ReLU(),
nn.BatchNorm2d(),
```

Dans cet exemple, le biais de la BatchNorm est appliqué après la fonction d'activation et n'a donc pas le même rôle que celui de la couche linéaire qui précède

## 3 Pooling

Les réseaux convolutionnels sont typiquement organisés par blocs de trois étapes :  
`convolution → activation → pooling`.

### 3.1 Introduction et motivations

L'application de couches convolutionnelles peut augmenter fortement la taille des données. Chaque filtre de convolution est appliqué de façon linéaire avant qu'une fonction d'activation soit appliquée sur la sortie. Ces deux applications sont parfois appelées **détection**. Le pooling intervient après ces deux opérations pour réduire la dimension de la sortie.

Une fonction de pooling effectue une sorte de sous-échantillonnage pour produire un "résumé" local du signal de sortie. Ce résumé peut être obtenu par différentes fonctions mathématiques décrites dans la section suivante.

Quelle que soit le type de fonction utilisée pour le pooling, son application rend la représentation plus invariante par rapport à de petites translations. C'est à dire que si on se déplace par une petite translation dans la donnée d'entrée, la sortie ne varie pas ou peu.

L'utilisation d'un pooling peut être assimilé à un a priori infini sur la fonction apprise par la couche pour que la fonction soit invariante à de petites translations.

### 3.2 Implémentation et taille de données

Comme les couches convolutionnelles, les couches de pooling ont des paramètres

- `kernel_size`
- `padding`
- `stride`
- `dilation`

Ainsi, selon les paramètres fixés, la taille des données de sortie peut être obtenue à partir de la taille des données d'entrée avec la formule suivante :

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - dilation[0] \times (kernel\_size[0] - 1) - 1}{stride[0]} + 1 \right\rfloor \quad (17)$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times padding[1] - dilation[1] \times (kernel\_size[1] - 1) - 1}{stride[1]} + 1 \right\rfloor \quad (18)$$

### 3.3 Différents types de pooling

Le pooling peut utiliser différentes fonctions dont les plus communes sont présentées dans les sections suivantes.

### 3.3.1 Max pooling

Le **max-pooling** consiste à sélectionner la valeur maximale sur un voisinage comme illustré par la figure 14.

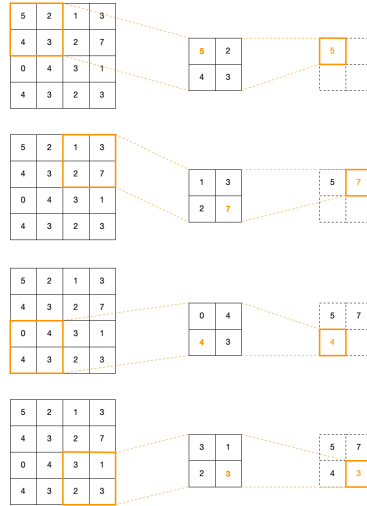


FIGURE 14 – Max-Pooling à 2 dimensions appliqué à une entrée de taille (4, 4).

### 3.3.2 Average pooling

L'**average-pooling** ou **mean-pooling** calcule la moyenne arithmétique des valeurs de la fenêtre ciblée. Ce pooling est illustré par la figure 15.

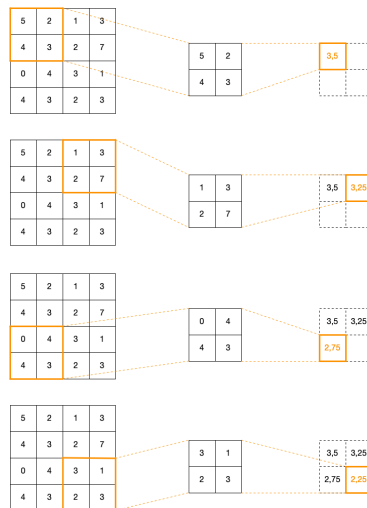


FIGURE 15 – Average-Pooling ou Mean-Pooling à 2 dimensions appliqué à une entrée de taille (4, 4).

### 3.3.3 Lp pooling

Le **Lp Pooling** calcule une moyenne après passage à la puissance  $p$  de toutes les valeurs sur la fenêtre de pooling. La formule de cette moyenne sur l'ensemble de valeurs  $X$  est la suivante :

$$f(X) = \sqrt[p]{\sum_{x \in X} x^p} \quad (19)$$

### 3.3.4 Pooling par réseau de neurones

La fonction de pooling appliquée sur chaque voisinage peut également prendre la forme d'un réseau de neurones. L'*attention pooling* est un exemple de ce pooling. Nous verrons dans le cours sur les Transformers le cas du *multi-head attention pooling*.

## 3.4 Pooling, une opération multi-dimensionnelle

Notez que lorsqu'un pooling est effectué sur un signal multi-dimensionnel et consiste à sous-échantillonner les données d'entrée. Ce pooling peut être opéré dans chacune des dimensions. La figure 16 présente le cas d'un pooling appris où la fonction prend la forme d'une moyenne pondérée dont les poids sont appris par un réseau de neurones. Selon la dimension dans laquelle la moyenne est calculée, la dimension de sortie et l'interprétation de ce pooling varient.

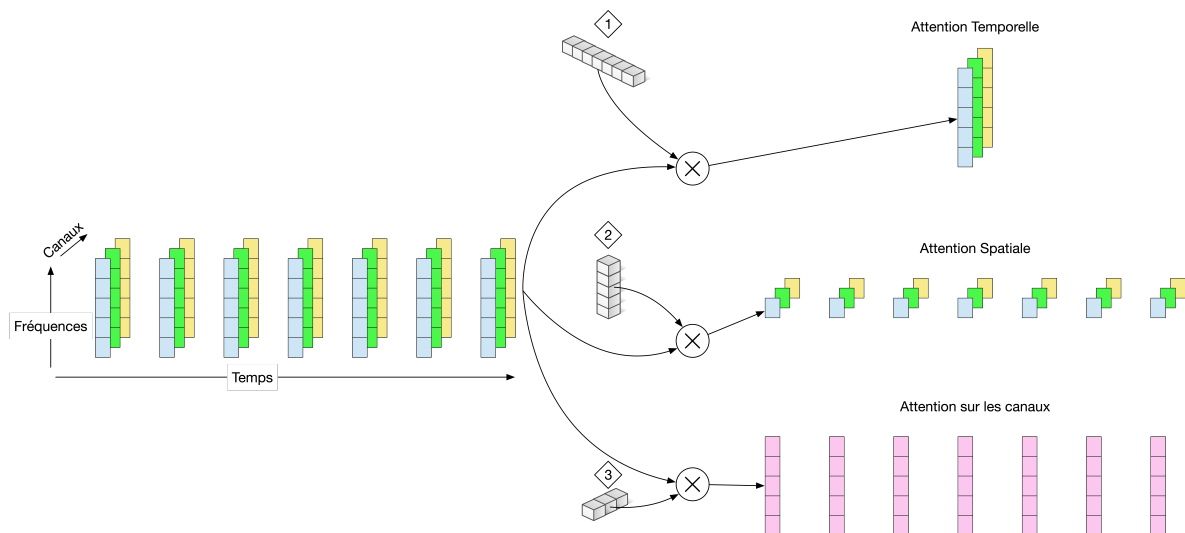


FIGURE 16 – Application d'un pooling appris par un réseau de neurones selon les 3 dimensions d'un signal d'entrée.

## 4 Exemple d'application

Dans cette section, nous illustrons dans un premier temps un exemple d'application concrète d'un réseau convolutionnel pour une tâche de classification d'images. Il s'agit donc d'un réseau convolutionnel à 2-dimensions. La seconde partie de cette section décrit brièvement plusieurs architectures classiques de réseaux de neurones convolutionnels pour des tâches de traitement de la parole ou d'images.

### 4.1 Classification d'images MNIST

Tâche : reconnaissance de digit sur la base de donnée MNIST (Modified National Institute of Standard and Technology).



FIGURE 17 – Extraits de la base de données MNIST.

Dans cette base de données, chaque image, de 28 par 28 pixels en niveaux de gris représente un chiffre manuscrit. Tâche de classification d'images de taille fixe.

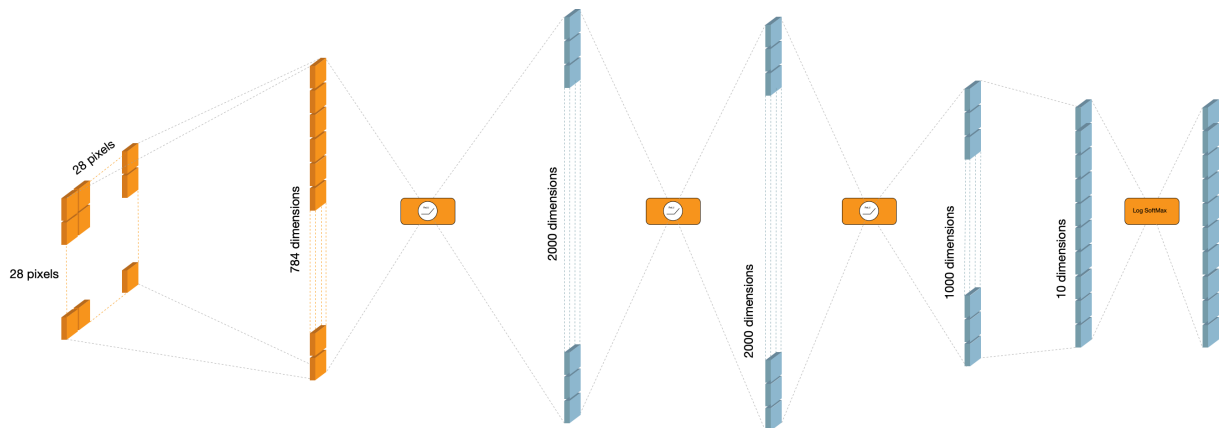


FIGURE 18 – Architecture utilisant des couches entièrement connectées pour la classification de chiffres manuscrits.

```
class MLP(torch.nn.Module):
    def __init__(self, num_inputs, num_classes, hidden_dims=(), activation='ReLU', **kwargs):
        super(MLP, self).__init__(**kwargs)
        self.layer_1 = torch.nn.Linear(784, 2000)
        self.layer_2 = torch.nn.Linear(2000, 2000)
        self.layer_3 = torch.nn.Linear(2000, 1000)
        self.layer_4 = torch.nn.Linear(1000, 10)
        self.flatten = nn.Flatten()
        self.activation = torch.nn.ReLU()
        self.logsoftmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.flatten(x)
        x = self.activation(self.layer_1(x))
        x = self.activation(self.layer_2(x))
        x = self.activation(self.layer_3(x))
        x = self.logsoftmax(self.layer_4(x))
        return x
```

Le code suivant propose une architecture convolutionnelle.

```
class CNN(torch.nn.Module):
    def __init__(self, num_classes, activation='ReLU'):
        super(CNN, self).__init__()
```

```
self.conv1 = torch.nn.Conv2d(1, 20, 5, 1)
self.conv2 = torch.nn.Conv2d(20, 50, 5, 1)
self.fc1 = torch.nn.Linear(800, 500)
self.fc2 = torch.nn.Linear(500, num_classes)
self.relu = torch.nn.ReLU()
self.pooling = torch.nn.MaxPool2d(2, stride=2)
self.logsoftmax = torch.nn.LogSoftmax(dim=1)

def forward(self, x):
    x = self.pooling(self.relu(self.conv1(x)))
    x = self.pooling(self.relu(self.conv2(x)))
    x = x.view(-1, 50 * 4 * 4)
    x = self.relu(self.fc1(x))
    x = self.fc2(x)
    x = self.logsoftmax(x)
    return x
```



## 4.2 Architectures classiques

Les réseaux convolutionnels obtiennent de très bonnes performances dans de nombreuses tâches. Cette section présente quelques unes des architectures populaires pour le traitement de la parole ou traitement d'image.

### 4.2.1 Temporal- Delay Neural Network (TDNN)

Les *Temporal- Delay Neural Network* ou TDNN ont été introduits pour la transcription de la parole **waibel1988speech**. TDNN

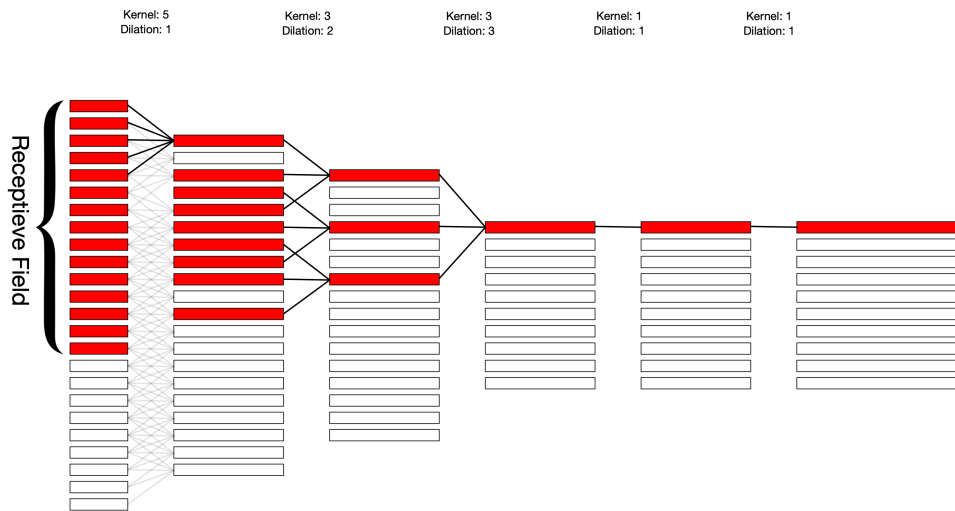


FIGURE 19 – Traitement d'une séquence par un modèle TDNN pour le traitement de la parole.

### 4.2.2 Temporal Convolutional Network (TCN)

Les *Temporal Convolutional Network* sont constitués de couches convolutionnelles causales. Ces réseaux sont utilisés pour la segmentation temporelle de signaux audio.

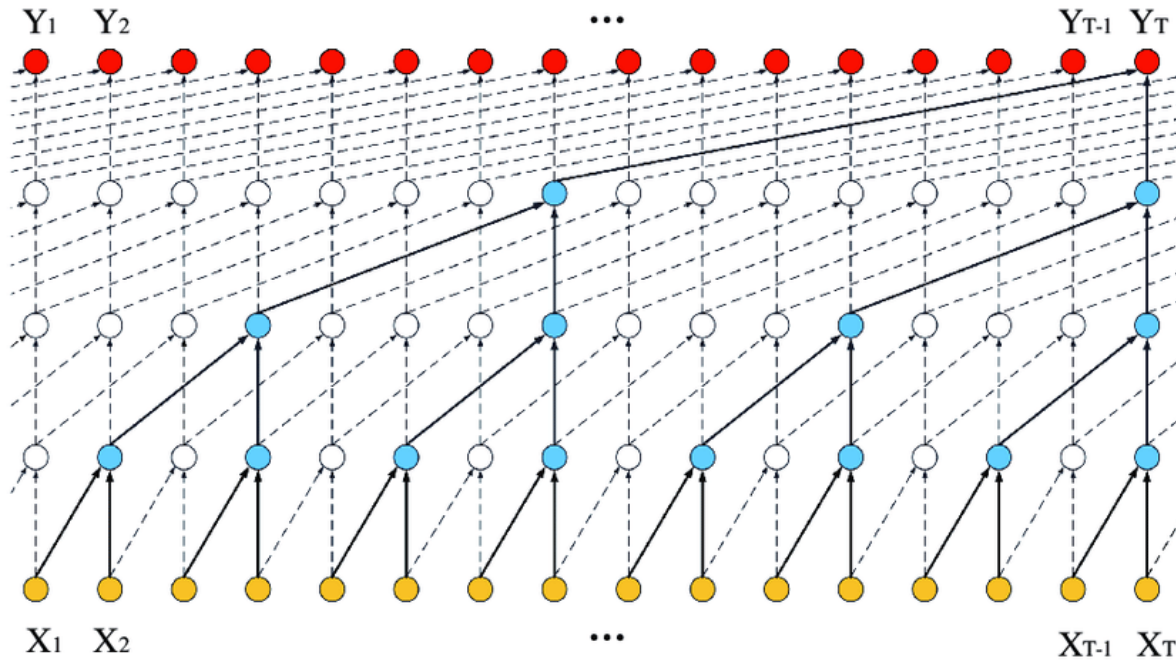


FIGURE 20 – Traitement d’une séquence par un modèle TCN pour le traitement de la parole.  
(vu le 03/10/24 sur <https://www.researchgate.net>)

### 4.2.3 AlexNet

AlexNet est un des premiers modèles qui a été appris en utilisant l’optimisation sur des cartes GPU. L’architecture est constituée de :

- 5 couches convolutionnelles suivie chacune d’une fonction d’activation de type ReLU
- 3 couches de max-pooling
- 2 couches de normalisation
- 2 couches entièrement connectées
- 1 SoftMax

La dimension de l’entrée est fixe en raison de l’utilisation de couches entièrement connectées et est :  $(224, 224, 3)$ . Le réseau compte 60 millions de paramètres.

#### 4.2.4 ResNet

Les ResNet ont été introduits pour le traitement d'images mais sont également utilisés pour la reconnaissance du locuteur. L'architecture décrite dans la figure 21 est utilisée dans le framework open-source SIDIAR développé au LIUM et est utilisé pour la reconnaissance du locuteur.

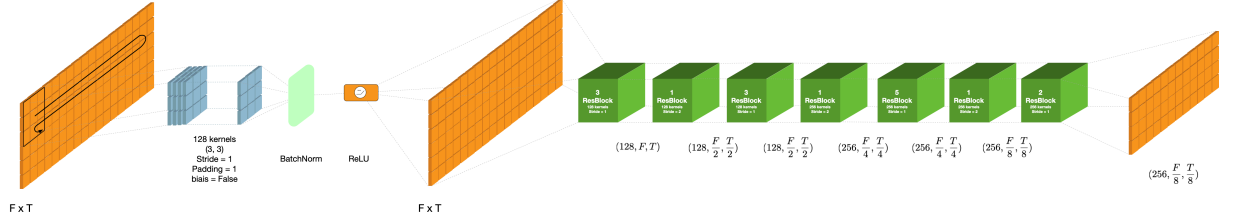


FIGURE 21 – Architecture d'un ResNet 34

Où chaque bloc vert sur le schéma est composé de 1, 3 ou 5 blocs dont l'architecture est décrite par la figure 22. Dans chaque bloc, la connection résiduelle inclut une couche convolutionnelle avec des filtre de dimension 1 qui permet de modifier la dimension des données d'entrée pour pouvoir les ajouter aux données qui sont transformées par le block.

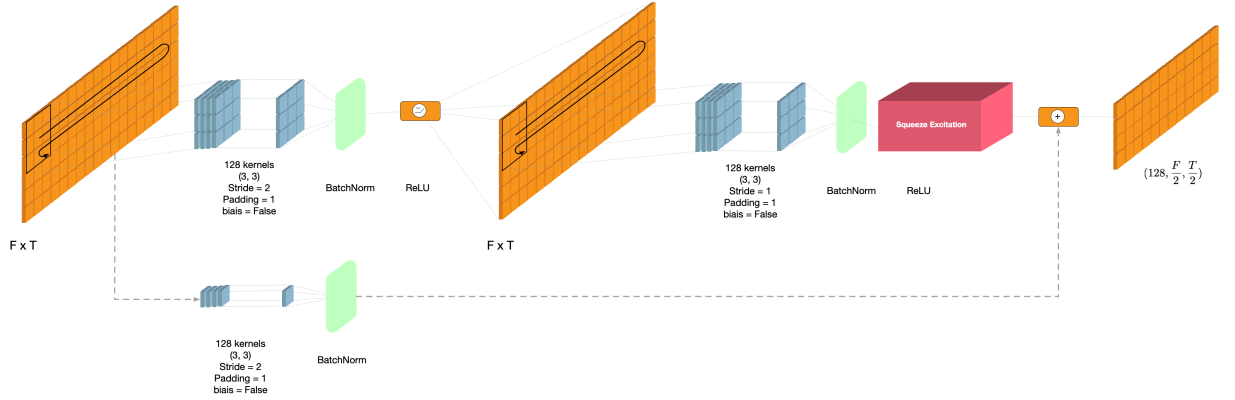


FIGURE 22 – Architecture d'un ResBlock avec 128 filtres de taille (3, 3), avec un padding de 1 et un stride de 2

Le code PyTorch correspondant à ce bloc est présenté ci-dessous :

```
class ResBlock(torch.nn.Module):
    """
    """
    expansion = 1

    def __init__(self, in_planes, planes, stride=1):
        super(ResBlock, self).__init__()
        self.conv1 = torch.nn.Conv2d(in_planes, planes, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = torch.nn.BatchNorm2d(planes)
```

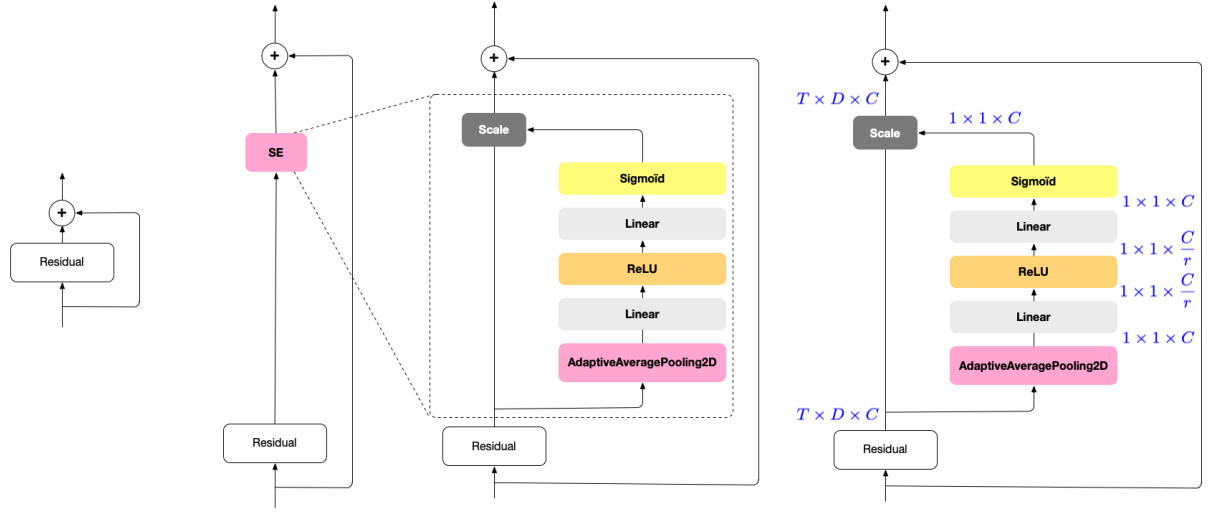
```
self.conv2 = torch.nn.Conv2d(planes, planes, kernel_size=3, stride=1, padding=1, bias=False)
self.bn2 = torch.nn.BatchNorm2d(planes)

self.se = SELayer(planes)

self.shortcut = torch.nn.Sequential()
if stride != 1 or in_planes != self.expansion*planes:
    self.shortcut = torch.nn.Sequential(
        torch.nn.Conv2d(in_planes, self.expansion * planes, kernel_size=1, stride=stride, bias=False),
        torch.nn.BatchNorm2d(self.expansion*planes)
    )

def forward(self, x):
    """
    :param x:
    :return:
    """
    out = torch.nn.functional.relu(self.bn1(self.conv1(x)))
    out = self.bn2(self.conv2(out))
    out = self.se(out)
    out += self.shortcut(x)
    out = torch.nn.functional.relu(out)
    return out
```

La Figure 23 illustre l'architecture de la *Squeeze Excitation Layer* qui agit comme une mécanisme d'auto-attention sur l'ensemble des canaux de convolution. Une couche de pooling adaptatif est appliquée sur les canaux d'entrée afin d'obtenir une unique valeur (scalaire) par canal, cette valeur est ensuite modifiée par deux couches successives pour produire un poids qui est ensuite multiplié par le canal correspondant.

FIGURE 23 – Détail d'une *Squeeze Excitation Layer*

Cette architecture sera rediscutée dans le cours sur les mécanismes d'attention.

#### 4.2.5 MobileNet

L'architecture MobileNet d'abord proposée dans [howard2017mobilenets](#) puis modifiée dans [howard2019searching](#) a été spécialement développée pour satisfaire aux contraintes des applications mobiles.

**4.2.5.1 DepthWise Separable Convolutions** Cette architecture de réseau utilise des convolutions *DepthWise Separables* pour réduire le nombre de paramètres nécessaire et ainsi le coût de calcul.

Pour illustrer ces convolutions, considérons un exemple où l'entrée est une image de dimension  $8 \times 8$  à 3 canaux. Afin de générer une sortie de même dimension à un seul canal, nous utilisons un filtre de dimension  $3 \times 3 \times 3$ . La convolution est illustrée par la figure 24.

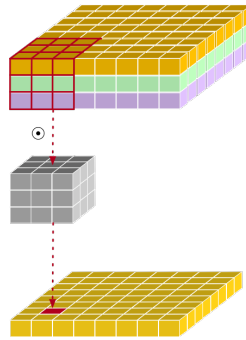


FIGURE 24 – Classic convolution to turn 3 channels into 1 single channel.

Le nombre de paramètres pour transformer 3 canaux en 1 seul est  $3 \times 3 = 9$ . Dans le cas où on transforme 3 canaux en 10 canaux de sortie, le nombre de paramètres sera  $3 \times 3 \times 10 = 90$ .

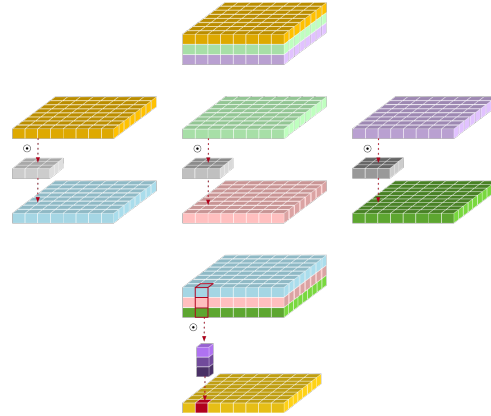


FIGURE 25 – Depth-wise separable convolution to turn 3 channels into 1 single channel.

L'utilisation de *Depthwise separable convolutions* permet d'accélérer grandement les calculs, réduire la mémoire, tout en ne dégradant que très peu les performances. Pour conserver les performances, il est possible d'augmenter le nombre de couches du réseau tout en conservant un coût de calcul moins élevé.

Le nombre de paramètres pour transformer 3 canaux en 1 seul est  $3 \times 3 + 3 = 12$ . Dans le cas où on transforme 3 canaux en 10 canaux de sortie, le nombre de paramètres sera  $3 \times 3 + 3 \times 10 = 39$  il est donc plus réduit que pour une convolution classique. Augmenter le nombre de paramètres peut causer un sur-apprentissage et la *depth-wise separable convolution* permet de limiter cet effet indésirable et d'accélérer le calcul.

Le code correspondant à cette combinaison de convolutions est présenté ci-dessous. On remarquera que la convolution *depthwise* est implémentée en utilisant l'option `group` du module de convolution de Pytorch, le détail peut être trouvé en annexe de ce document.

```
import torch

class depthwise_separable_conv(nn.Module):
    def __init__(self, nin, nout):
        super(depthwise_separable_conv, self).__init__()
        self.depthwise = torch.nn.Conv2d(nin, nin, kernel_size=3, padding=1, groups=nin)
        self.pointwise = torch.nn.Conv2d(nin, nout, kernel_size=1)

    def forward(self, x):
        out = self.depthwise(x)
        out = self.pointwise(out)
        return out
```

**4.2.5.2 Inverted residual blocks** En règle général, et comme nous l'avons vu dans l'exemple du ResNet précédemment, les réseaux convolutionnels utilisent des blocs résiduels de type :

$$\text{wide} \rightarrow \text{narrow} \rightarrow \text{wide} \quad (20)$$

comme illustré sur la figure 26. La compression des données avec une convolution *pointwise*, peu coûteuse, permet de réaliser une convolution sur un nombre réduit de paramètres avant de rétablir le nombre de canaux originaux pour combiner cette information avec l'entrée du bloc résiduel.

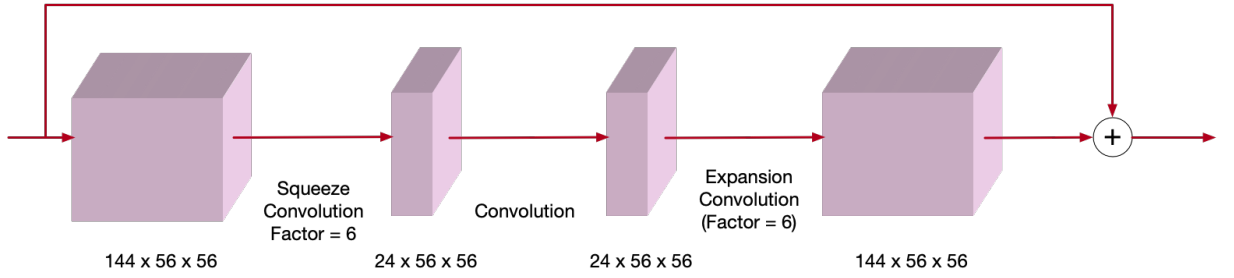


FIGURE 26 – Classic residual block architecture, wide  $\rightarrow$  narrow  $\rightarrow$  wide.

À partir de MobileNetV2 les auteurs utilisent des blocs convolutionnels résiduels inversés. À l'inverse des blocs classiques, qui prennent en entrée des données avec de nombreux canaux, qui compressent ces informations à l'intérieur du bloc résiduel puis ré-augmentent le nombre de canaux en sortie du bloc.

$$\text{narrow} \rightarrow \text{wide} \rightarrow \text{narrow} \quad (21)$$

Les blocs convolutionnels résiduels inversés prennent une entrée contenant peu de canaux, augmentent le nombre de canaux, utilisent alors une convolution *depthwise separable* sur ce grand nombre de canaux avant de réduire le nombre de canaux pour la sortie du bloc.

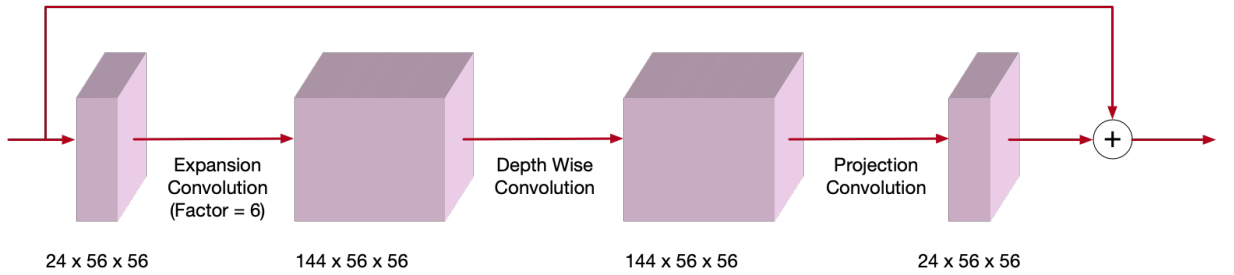


FIGURE 27 – Inverted residual block architecture, narrow  $\rightarrow$  wide  $\rightarrow$  narrow.

Comme les convolutions *DepthWise separables* diminuent grandement le nombre de paramètres et la quantité de calcul, le bloc inversé peut appliquer cette convolution sur un grand nombre

de canaux sans augmenter le coût de calcul de façon dramatique. L'augmentation du nombre de canaux est faite avec une convolution *pointwise* qui est peu coûteuse. Le nombre de canaux est réduit avant la sortie du bloc résiduel pour limiter la dimension des données qui transitent dans le réseau.

**Remarque :**

Notez que chaque couche convolutionnelle est suivie d'une **BatchNorm** et d'une activation (**ReLU** dans le modèle MobileNet et **ReLU6** pour MobileNetV2. Seules les couches de sortie des blocs résiduelles ne sont pas suivies de fonction d'activation, comme décrit juste après.

**4.2.5.3 Utilisation d'une couche linéaire dans le bloc résiduel** Une autre modification apportée dans la version 2 du MobileNet consiste à supprimer la fonction d'activation après la dernière couche du bloc convolutionnel résiduel inversé. Comme le nombre de canaux en sortie du bloc convolutionnel est faible, les auteurs de l'article original ont observé qu'utiliser une non-linéarité à cet endroit réduit la quantité d'information qui est apportée par le bloc convolutionnel.

Le **MobileNetV2** est composé de blocs résiduels convolutionnels ressemblant à celui illustré par la figure 28, le choix des paramètres de ce bloc :

- pas de stride,  $s$
- facteur d'expansion,  $t$
- taille des filtres de la convolution *depth-wise*
- facteur de projection

détermine la taille de la sortie de ce bloc et varie selon la place du bloc dans le réseau. Ces blocs sont appelés *bottle-neck* en raison de la réduction du nombre de canaux qui est réalisé à la sortie.

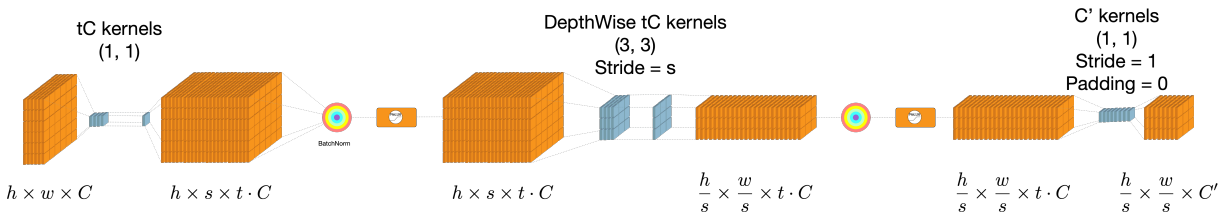


FIGURE 28 – Architecture d'un bloc résiduel convolutionnel bottle-neck utilisé dans les MobileNetV2.

L'architecture complète du **MobileNetV2**, introduit dans [sandler2018mobilenetv2](#) est illustrée par le tableau 1 et un code Pytorch correspondant est proposé ci dessous.

```
_BOTTLENECK_EXPANSION = 4
```



| Dimensions d'entrée      | Opérateur               | t | c    | n | s |
|--------------------------|-------------------------|---|------|---|---|
| $224^2 \times 3$         | conv2d                  | - | 32   | 1 | 2 |
| $112^2 \times 32$        | bottleneck              | 1 | 16   | 1 | 1 |
| $112^2 \times 16$        | bottleneck              | 6 | 24   | 2 | 2 |
| $56^2 \times 24$         | bottleneck              | 6 | 32   | 3 | 2 |
| $28^2 \times 32$         | bottleneck              | 6 | 64   | 4 | 2 |
| $14^2 \times 64$         | bottleneck              | 6 | 96   | 3 | 1 |
| $14^2 \times 96$         | bottleneck              | 6 | 160  | 3 | 2 |
| $7^2 \times 160$         | bottleneck              | 6 | 320  | 1 | 1 |
| $7^2 \times 320$         | conv2d $1 \times 1$     | - | 1280 | 1 | 1 |
| $7^2 \times 1280$        | AvgPooling $7 \times 7$ | - | -    | 1 | - |
| $1 \times 1 \times 1280$ | conv2d $1 \times 1$     | - | k    | - | - |

TABLE 1 – Architecture MobileNetV2.  $t$  est le facteur d'expansion des blocs Bottleneck,  $c$  le nombre de canaux de sortie,  $n$  le nombre de fois que le bloc est répété et  $s$  le pas de stride de la première couche du bloc. Les couches suivantes ont un stride de 1.

```

class _ConvBnReLU(torch.nn.Module):
    """
    Convolutional block combining convolutional layer, batchnorm and activation if needed
    """

    def __init__(self, in_ch, out_ch, kernel_size, to_stride, padding, dilation, linear=True):
        super(_ConvBnReLU, self).__init__()

        self.conv = torch.nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding, dilation, bias=False)
        self.bn = torch.nn.BatchNorm2d(out_ch, eps=1e-5, momentum=1 - 0.999)
        self.activation = torch.nn.ReLU6()
        self.linear = linear

    def forward(self, x):
        x = self.bn(self.conv(x))
        if not linear:
            x = self.activation(x)
        return x

class _Stem(torch.nn.Module):
    """
    Input convolutional layer
    """

    def __init__(self, out_ch):
        super(_Stem, self).__init__()
        self.conv_bn_block = _ConvBnReLU(3, out_ch, 7, 2, 3, 1)

```

```
self.pooling = torch.nn.MaxPool2d(3, 2, 1, ceil_mode=True)

def forward(self, x):
    x = self.conv_bn_block(x)
    x = self.pooling(x)
    return x

class _Bottleneck(nn.Module):
    """
    Bottleneck block
    """

    def __init__(self, in_ch, out_ch, stride, dilation, downsample):
        super(_Bottleneck, self).__init__()
        mid_ch = out_ch // _BOTTLENECK_EXPANSION
        self.reduce = _ConvBnReLU(in_ch, mid_ch, 1, stride, 0, 1, True)
        self.conv3x3 = _ConvBnReLU(mid_ch, mid_ch, 3, 1, dilation, dilation, True)
        self.increase = _ConvBnReLU(mid_ch, out_ch, 1, 1, 0, 1, False)
        self.shortcut = (
            _ConvBnReLU(in_ch, out_ch, 1, stride, 0, 1, False)
            if downsample
            else nn.Identity()
        )

    def forward(self, x):
        h = self.reduce(x)
        h = self.conv3x3(h)
        h = self.increase(h)
        h += self.shortcut(x)
        return F.relu(h)

class _ResLayer(nn.Sequential):
    """
    Residual layer with multi grids
    """

    def __init__(self, n_layers, in_ch, out_ch, stride, dilation, multi_grids=None):
        super(_ResLayer, self).__init__()

        if multi_grids is None:
            multi_grids = [1 for _ in range(n_layers)]
        else:
            assert n_layers == len(multi_grids)
```

---

```

    # Downsampling is only in the first block
    for i in range(n_layers):
        self.add_module(
            "block{}".format(i + 1),
            _Bottleneck(
                in_ch=(in_ch if i == 0 else out_ch),
                out_ch=out_ch,
                stride=(stride if i == 0 else 1),
                dilation=dilation * multi_grids[i],
                downsample=(True if i == 0 else False),
            ),
        )

class _ASPP(nn.Module):
    """
    Atrous spatial pyramid pooling (ASPP)
    """

    def __init__(self, in_ch, out_ch, rates):
        super(_ASPP, self).__init__()
        for i, rate in enumerate(rates):
            self.add_module(
                "c{}".format(i),
                nn.Conv2d(in_ch, out_ch, 3, 1, padding=rate, dilation=rate, bias=True),
            )

        for m in self.children():
            nn.init.normal_(m.weight, mean=0, std=0.01)
            nn.init.constant_(m.bias, 0)

    def forward(self, x):
        return sum([stage(x) for stage in self.children()])

class DeepLabV2(nn.Sequential):
    """
    DeepLab v2: Dilated ResNet + ASPP
    Output stride is fixed at 8
    """

    def __init__(self, n_classes, n_blocks, atrous_rates):
        super(DeepLabV2, self).__init__()
        ch = [64 * 2 ** p for p in range(6)]
        self.add_module("layer1", _Stem(ch[0]))

```

```

self.add_module("layer2", _ResLayer(n_blocks[0], ch[0], ch[2], 1, 1))
self.add_module("layer3", _ResLayer(n_blocks[1], ch[2], ch[3], 2, 1))
self.add_module("layer4", _ResLayer(n_blocks[2], ch[3], ch[4], 1, 2))
self.add_module("layer5", _ResLayer(n_blocks[3], ch[4], ch[5], 1, 4))
self.add_module("aspp", _ASPP(ch[5], n_classes, atrous_rates))

def freeze_bn(self):
    for m in self.modules():
        if isinstance(m, _ConvBnReLU.bn):
            m.eval()

```

#### 4.2.6 U-Net

Proposé pour la segmentation d'images bio-médicales dans [ronneberger2015u](#) et utilisés depuis dans de nombreuses autres applications, l'architecture U-Net ne contient que des couches convolutionnelles sans aucune couche complètement connectée. L'architecture U-Net est composée de deux types de blocs appelés Down et Up. Les blocs Down effectuent un sous-échantillonnage avec *max-pooling* suivi de deux convolutions comme décrit par la figure 29.

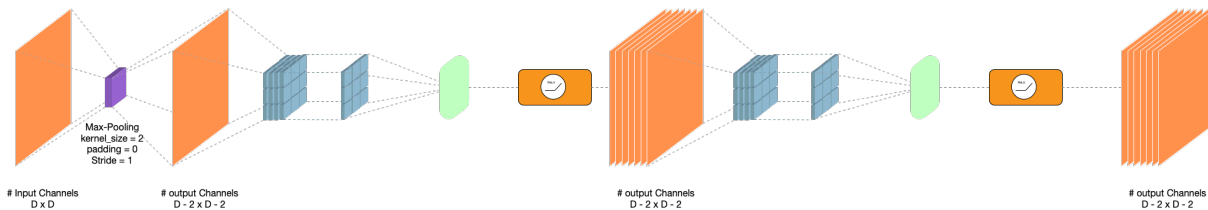


FIGURE 29 – Bloc *Down* de l'architecture U-Net

Le code de ce module est le suivant :

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class DoubleConv(nn.Module):
    """(convolution => [BN] => ReLU) * 2"""

    def __init__(self, in_channels, out_channels, mid_channels=None):
        super().__init__()
        if not mid_channels:
            mid_channels = out_channels
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channels, mid_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(mid_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(mid_channels, out_channels, kernel_size=3, padding=1),

```

```

        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True)
    )

    def forward(self, x):
        return self.double_conv(x)

class Down(nn.Module):
    """Downscaling with maxpool then double conv"""

    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.maxpool_conv = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_channels, out_channels)
        )

    def forward(self, x):
        return self.maxpool_conv(x)

```

Les bloc Up effectuent eux un sur-échantillonnage qui permet d'améliorer la résolution de la segmentation tout en combinant des informations contextuelles de différents niveaux.

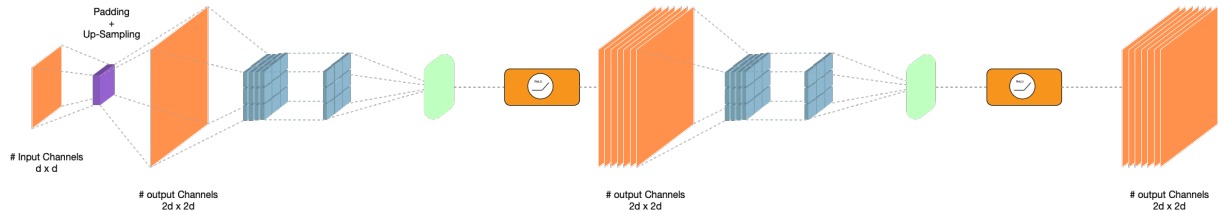


FIGURE 30 – Bloc *Up* de l'architecture U-Net

```

class Up(nn.Module):
    """Upscaling then double conv"""

    def __init__(self, in_channels, out_channels, bilinear=True):
        super().__init__()

        # if bilinear, use the normal convolutions to reduce the number of channels
        if bilinear:
            self.up = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
            self.conv = DoubleConv(in_channels, out_channels, in_channels // 2)
        else:
            self.up = nn.ConvTranspose2d(in_channels, in_channels // 2, kernel_size=2, stride=2)
            self.conv = DoubleConv(in_channels, out_channels)

```

```
def forward(self, x1, x2):
    x1 = self.up(x1)
    # input is Channel x Height x Width
    diffY = x2.size()[2] - x1.size()[2]
    diffX = x2.size()[3] - x1.size()[3]

    x1 = F.pad(x1, [diffX // 2, diffX - diffX // 2,
                   diffY // 2, diffY - diffY // 2])
    x = torch.cat([x2, x1], dim=1)
    return self.conv(x)
```

L'architecture complète est décrite par la figure 31 et le code de ce module est le suivant :

```
""" Full assembly of the parts to form the complete network """
```

```
import torch.nn.functional as F

from .unet_parts import *

class UNet(nn.Module):
    def __init__(self, n_channels, n_classes, bilinear=True):
        super(UNet, self).__init__()
        self.n_channels = n_channels
        self.n_classes = n_classes
        self.bilinear = bilinear

        self.inc = DoubleConv(n_channels, 64)
        self.down1 = Down(64, 128)
        self.down2 = Down(128, 256)
        self.down3 = Down(256, 512)
        factor = 2 if bilinear else 1
        self.down4 = Down(512, 1024 // factor)
        self.up1 = Up(1024, 512 // factor, bilinear)
        self.up2 = Up(512, 256 // factor, bilinear)
        self.up3 = Up(256, 128 // factor, bilinear)
        self.up4 = Up(128, 64, bilinear)
        self.outc = OutConv(64, n_classes)

    def forward(self, x):
        x1 = self.inc(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)
        x5 = self.down4(x4)
        x = self.up1(x5, x4)
        x = self.up2(x, x3)
        x = self.up3(x, x2)
        x = self.up4(x, x1)
        logits = self.outc(x)
        return logits
```

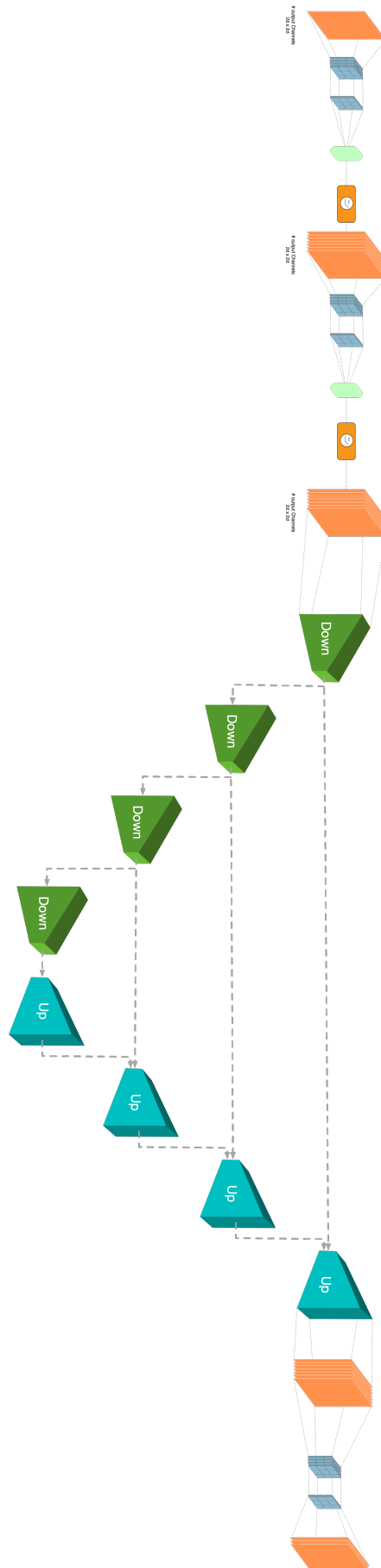


FIGURE 31 – Vue d'ensemble de l'architecture U-Net



## Annexes 1 : Convolutions, groupes et Depthwise

### 4.3 Convolution par groupes

L'option `groups` des modules de convolution de Pytorch permettent de modifier le comportement des couches convolutionnelles, et en particulier de reproduire le comportement des convolutions *depthwise* qui sont utilisées pour approximer des convolutions entières et accélérer grandement le calcul comme dans les modèles MobileNet par exemple.

L'implémentation de cette option est donnée par le code suivant en Numpy.

```
import numpy as np
import torch

def pad_image(filters, image):

    # ne fonctionne quye pour des filtres carrés
    assert filters.shape[0] == filters.shape[1]
    # et des filtres de dimension paire
    assert filters.shape[0] % 2

    filter_len = filters.shape[0]
    pad = filter_len // 2

    image_padded = np.pad(image, [(pad,), (pad,), (0,)], 'constant')

    return filter_len, image_padded

def convert_to_torch(image, filters):

    image_torch = torch.tensor(image.transpose([2, 1, 0])[None])
    filters_torch = torch.tensor(filters.transpose([3, 2, 1, 0]))

    return image_torch, filters_torch

def cnn2d_grouped(image: np.ndarray,
                  filters: np.ndarray,
                  groups: int = 1) -> np.ndarray:
    """
    Grouped convolutions. Can emulate depthwise convolutions but has bit more flexibility.
    Args:
        image: (hi, wi, cin).
        filters: (hf, wf, cin / groups, cout).
```

## 4 EXEMPLE D'APPLICATION

---

```
groups: must divide cin and cout.

Returns:
    (hi, wi, cout)
"""

# filters are divided into groups so the number must be a multiple of groups
assert filters.shape[-1] % groups == 0
# input channels are processed by a group of filter so it must be a multiple
assert image.shape[-1] == groups * filters.shape[-2]

# effectue le padding pour conserver la dimension de l'image
filter_len, image_padded = pad_image(filters, image)

# Initialise la sortie aux bonnes dimensions (mêmes que les dimensions d'entrée au nombre de canaux près)
out = np.zeros([image.shape[0], image.shape[1], filters.shape[-1]])

# Calcule le nombre de canaux d'entrée de chaque groupe
n_cin_group = image.shape[-1] // groups
# Calcule le nombre de canaux de sortie de chaque groupe
n_cout_group = filters.shape[-1] // groups

# Pour chaque groupe, chaque pixel et chaque canal de sortie de ce groupe, calcule la convolution
for idx_group in range(groups):
    for idx_row in range(image.shape[0]):
        for idx_col in range(image.shape[1]):
            for idx_cout_group in range(n_cout_group):

                idx_cout = n_cout_group * idx_group + idx_cout_group

                filter = filters[:, :, :, idx_cout]
                patch = image_padded[idx_row: idx_row + filter_len,
                                     idx_col: idx_col + filter_len,
                                     idx_group * n_cin_group: (idx_group + 1) * n_cin_group]

                out[idx_row, idx_col, idx_cout] = (filter * patch).sum()

return out

def cnn2d_grouped_torch(image: np.ndarray,
                        filters: np.ndarray,
                        groups: int = 1) -> np.ndarray:

    from torch.nn import functional as F
```

```
image_torch, filters_torch = convert_to_torch(image, filters)

features_torch = F.conv2d(image_torch, filters_torch, padding=filters.shape[0] // 2, groups=groups)
features_torch_ = features_torch.numpy()[0].transpose([2, 1, 0])

return features_torch_
```

#### 4.4 Implémenter une convolution *Depthwise*

```
import numpy as np

from cnns.utils import pad_image, convert_to_torch

def cnn2d_depthwise(image: np.ndarray,
                    filters: np.ndarray):
    """
    Depthwise convolutions.
    Args:
        image: (hi, wi, cin).
        filters: (hf, wf, cin, cmul).

    Returns:
        (hi, wi, cin * cmul)
    """

    filter_len, image_padded = pad_image(filters, image)

    n_cout_cin = filters.shape[-1]

    out = np.zeros([image.shape[0], image.shape[1], image.shape[2] * n_cout_cin])

    for idx_cin in range(image.shape[2]):
        for idx_row in range(image.shape[0]):
            for idx_col in range(image.shape[1]):
                patch = image_padded[idx_row: idx_row + filter_len, idx_col: idx_col + filter_len, idx_cin]
                for idx_cmul in range(n_cout_cin):
                    filter = filters[:, :, idx_cin, idx_cmul]
                    out[idx_row, idx_col, idx_cin * n_cout_cin + idx_cmul] = (filter * patch).sum()

    return out

def cnn2d_depthwise_torch(image: np.ndarray,
                          filters: np.ndarray):
```

```
from torch.nn import functional as F

image_torch, filters_torch = convert_to_torch(image, filters)

df, _, cin, cmul = filters.shape
filters_torch = filters_torch.transpose(0, 1).contiguous()
filters_torch = filters_torch.view(cin * cmul, 1, df, df)

features_torch = F.conv2d(image_torch, filters_torch, padding=df // 2, groups=cin)
features_torch_ = features_torch.numpy()[0].transpose([2, 1, 0])

return features_torch_
```

## 4.5 Implémenter une convolution *Depthwise Separable*

```
import torch

class depthwise_separable_conv(nn.Module):
    def __init__(self, nin, nout):
        super(depthwise_separable_conv, self).__init__()
        self.depthwise = torch.nn.Conv2d(nin, nin, kernel_size=3, padding=1, groups=nin)
        self.pointwise = torch.nn.Conv2d(nin, nout, kernel_size=1)

    def forward(self, x):
        out = self.depthwise(x)
        out = self.pointwise(out)
        return out
```

